

Part 1: Hash and salt

Ethical and authorized use: enter only fictional passwords made for this lab. Never place a real password in source code, shell history, or screenshots.

Run `python3 hash_password.py` twice with the same input, then run `python3 salted_hash.py` twice. SHA-256 is one-way hashing, not encryption: there is no decryption key. Observe that unsalted digests repeat, while random salts change the demonstration verifier.

These constructions teach mechanics only. Neither raw SHA-256 nor `SHA256(salt || password)` is appropriate production password storage.

Run the examples

Start in this directory inside the course container:

```
cd /workspace/labs/lab01/part1_hashing
```

First, hash the same fictional password twice. `getpass` deliberately does not display the password while you type it.

```
python3 hash_password.py
Educational demo only; SHA-256 alone is unsafe for password storage.
Password:
<64 hexadecimal characters>
```

Run the command again with the same input. The two digests should be identical because SHA-256 is deterministic.

Next, run the salted demonstration twice with the same fictional password:

```
python3 salted_hash.py
Password:
salt=<32 hexadecimal characters>
digest=<64 hexadecimal characters>
Educational demo only; use a password KDF in production.
```

The salt and digest should normally change on every run. The salt is stored alongside the verifier; it is unique, not secret.

Run the focused tests:

```
pytest -q test_hashing.py
```

Expected result: 2 passed.

Checkpoint questions

1. Why is encryption, whose design permits decryption, the wrong default model for password verification?
2. Why do identical inputs produce identical unsalted SHA-256 digests?
3. What does a unique salt change, and what does it not prevent?